Project Technical Architecture and API Guide

Smart System for Analyzing and Predicting Electricity Consumption

This guide explains how the browser application, Express API, MySQL database, and prediction service work together. It follows the implemented routes and data transformations, including both calendar-based daily readings and weekly summary records. The key design is a same-origin JSON API: the browser calls Express, Express owns authentication and persistence, and only the prediction path calls the separate FastAPI service.

System at a glance

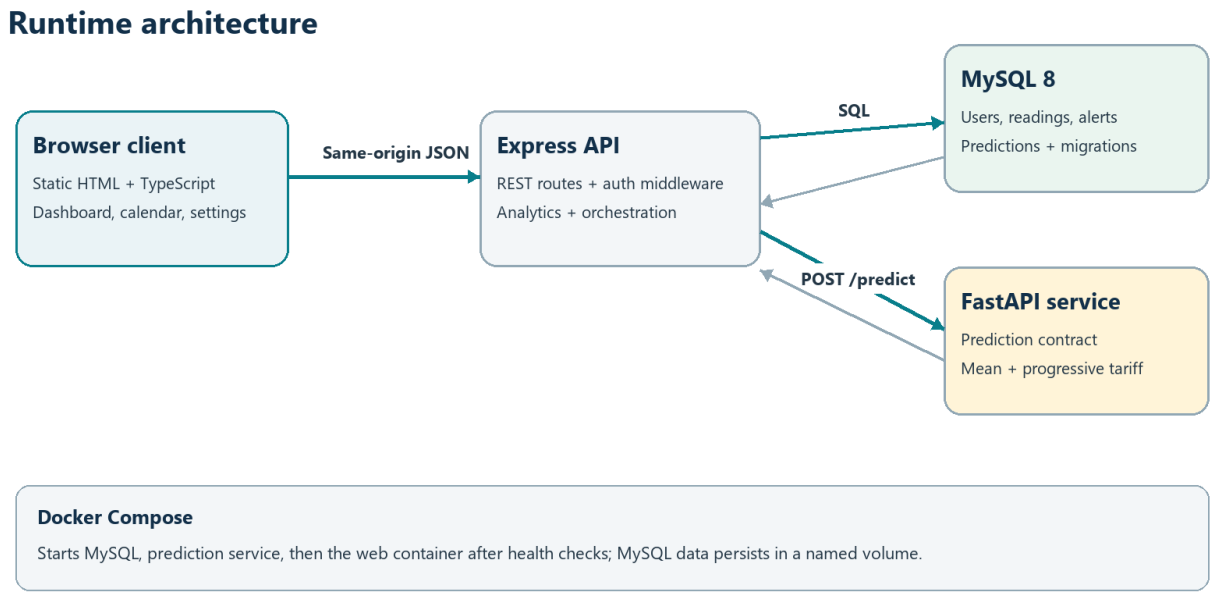


Figure 1. The browser communicates with the Express API. The API reads and writes MySQL and calls FastAPI only when a forecast is requested.

Main components

Component	Responsibility
Browser client	Static HTML and TypeScript pages for sign in, registration, dashboard, calendar, settings, charts, and API calls.
Express API	Serves the REST endpoints, validates requests, verifies JWTs, scopes data to the current user, computes dashboard analytics, and orchestrates prediction requests.
MySQL 8	Stores accounts, daily and weekly consumption, alerts, and forecast results. TypeORM migrations are the schema upgrade path; schema synchronization is disabled.
FastAPI prediction service	Accepts weekly usage and tariff inputs, returns a four-week estimate, estimated bill, model label, and error range.

Request lifecycle

At application startup, Express initializes the TypeORM data source and runs pending migrations before continuing requests. Docker Compose also runs compiled migrations before starting the web process. The browser and API share the site origin, so the client calls relative paths such as /api/dashboard without configuring a separate browser-facing API host.

User interactions and request sequences

The browser’s api() helper serializes request bodies as JSON, adds the stored bearer token when present, parses JSON responses, and turns non-success responses into visible errors. If an authenticated request receives 401, it clears the local token and sends the browser to the sign-in page.

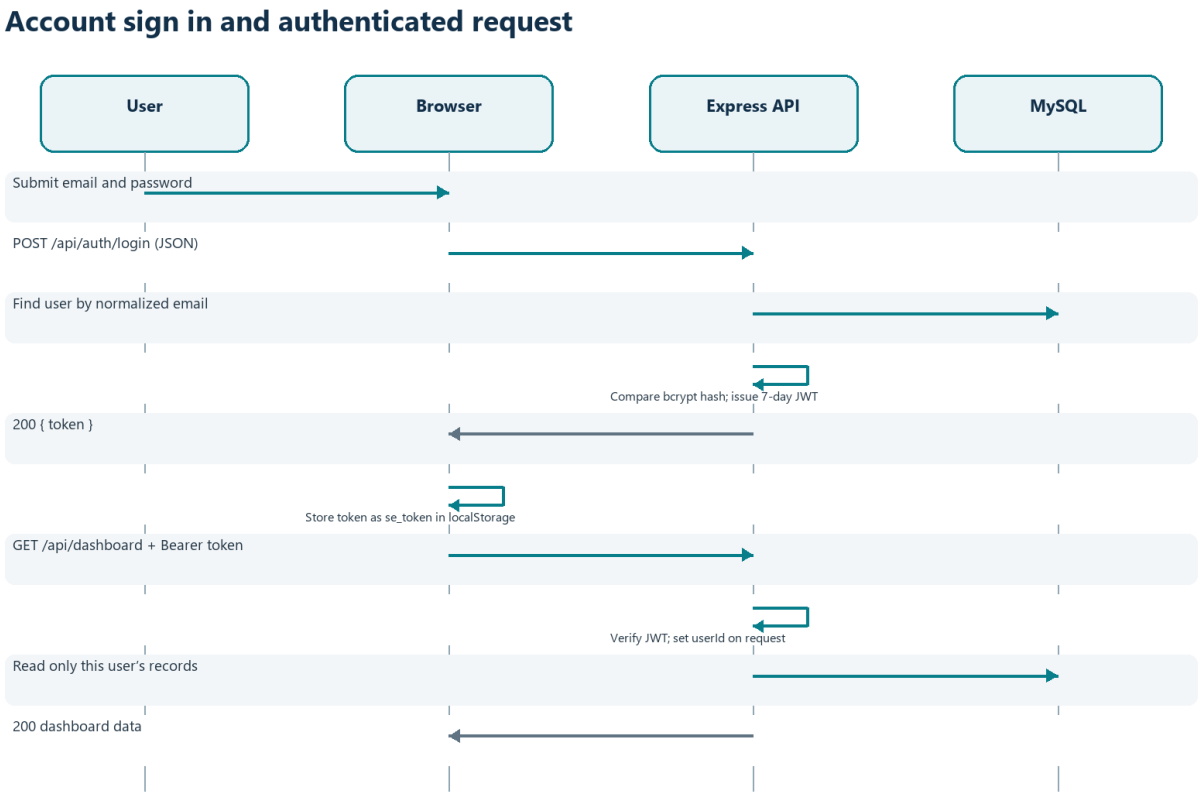


Figure 2. Sign-in produces a seven-day JWT. Later API calls attach it as a bearer token.

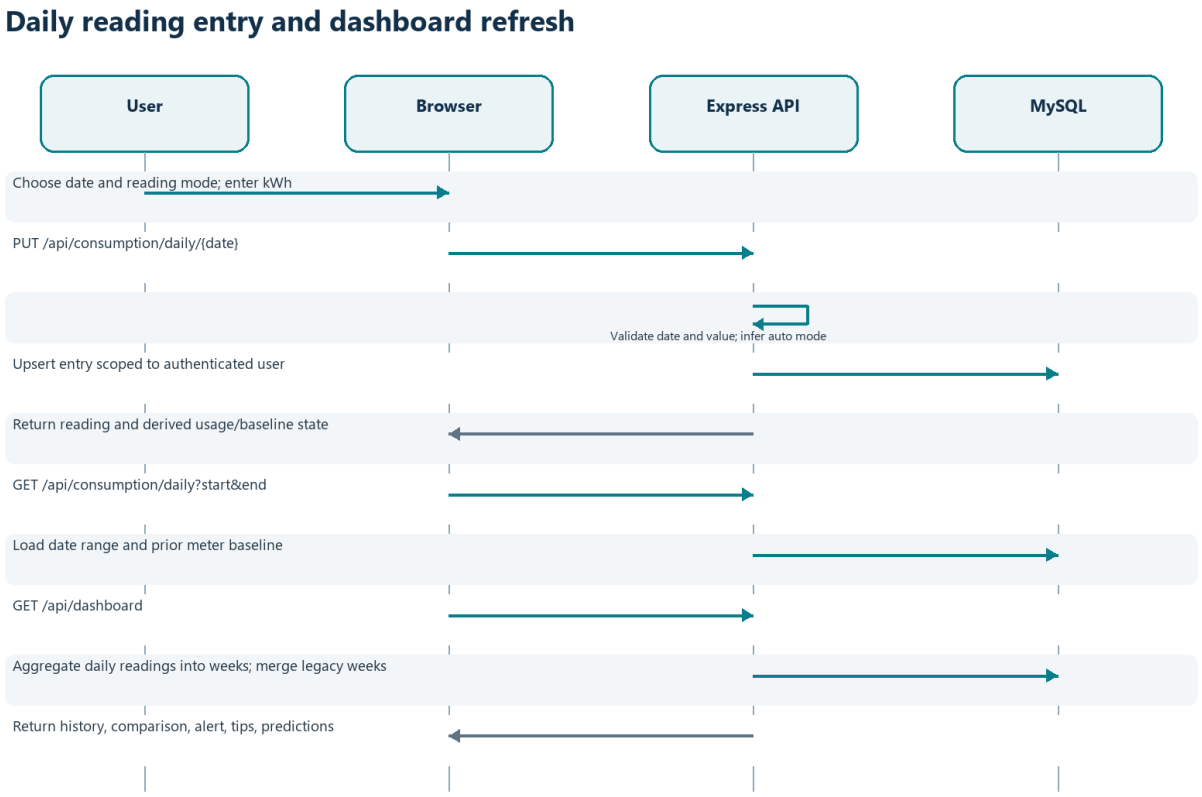


Figure 3. Daily entries are upserted; dashboard history is then rebuilt from readings and any remaining weekly summaries.

For cumulative meter totals, the first total establishes a baseline and does not contribute derived usage until another total is available. A later total is differenced against the previous total. The API rejects a total lower than its prior total or higher than its next total. In auto mode, a value at or above the preceding total is classified as a meter total; otherwise it is treated as daily use. When there is no earlier meter total, auto mode defaults to daily use.

REST API reference

All API routes use the /api prefix. JSON request and response bodies are used throughout. Routes under /api/consumption and the insights router require Authorization: Bearer <token>; register and login are public. The API accepts JSON request bodies up to 10 KB.

Authentication

Method and path	Request	Success	Common errors
POST /api/auth/register	{ email, password, fullName? }	201 { token }	400 invalid fields; 409 email already registered
POST /api/auth/login	{ email, password }	200 { token }	400 missing fields; 401 invalid credentials

Email is normalized to lowercase. Passwords must be at least eight characters and are stored as bcrypt hashes (cost 10). Authentication endpoints are limited to 10 requests per 15 minutes. The same login error covers an unknown email and a wrong password.

Daily consumption and calendar

Method and path	Request / query	Success	Behavior
GET /api/consumption/daily	?start=YYYY-MM-DD&end=YYYY-MM-DD	200 array of { date, reading, readingType, units, derived, … }	Loads range; includes previous meter-total baseline when available.
PUT /api/consumption/daily/{date}	{ units, readingType? }	200 { date, reading, readingType, detected, units, derived, previousReadingDate, needsBaseline }	Upserts the user/date entry. Mode: auto, daily_use, or meter_total.
DELETE /api/consumption/daily/{date}	No body	204 empty	Deletes that user's reading; 404 if absent.

Date range parameters must be valid calendar dates with start on or before end. Units must be finite and non-negative. The UI refreshes the calendar and dashboard after a successful change.

Weekly summary records

Method and path	Request / query	Success	Behavior
GET /api/consumption	Optional paired start and end dates	200 [{ weekStart, units, billAmount }]	Oldest first; filtering uses Monday week_start.
POST /api/consumption	{ date, units, billAmount? }	201 { weekStart, units }	Maps date to Monday. Same user/week is inserted or updated.
PUT /api/consumption/{weekStart}	{ units, billAmount? }	200 { weekStart, units, billAmount }	Updates existing Monday record; 404 if absent.
DELETE /api/consumption/{weekStart}	No body	204 empty	Deletes existing record; 404 if absent.

Weekly inputs require a valid date; explicit update/delete keys must be Mondays. Units and any supplied bill amount must be non-negative. Every record lookup or mutation is restricted by authenticated user ID.

Dashboard, account, and prediction APIs

Dashboard and alerts

Method and path	Success payload	Notes
GET /api/dashboard	{ history, averageWeeklyUnits, comparison, alert, alerts, predictions, tips, invoiceDay }	History is sorted by week. Comparison covers the latest two weeks; stored alerts and forecasts are capped at 20 and 5.
GET /api/dashboard/monthly	{ months, comparison, invoiceDay, weekMapping }	Groups weeks into invoice cycles based on Monday week_start and the account invoice day.
GET /api/alerts	Array of { id, type, message, createdAt }	Latest 20 stored alerts.

Account profile and tariff

Method and path	Request / response	Validation and behavior
GET /api/me	Response includes fullName, email, tariffMode, invoiceDay, pricePerKwh, tierLimitKwh, baseRatePerKwh, highRatePerKwh.	pricePerKwh is null when tiered mode is active.
PATCH /api/me	{ fullName, tariffMode, invoiceDay?, pricePerKwh? }	Name 1–100 chars; invoice day 1–28; mode tiered or flat_override; flat rate >0 and ≤5 SAR/kWh.

Forecast

Method and path	Request	Success	Errors
POST /api/predict	No client body required; bearer token required.	200 { predictionMonth, predictedUnits, predictedBill, weeklyUnits, lowerUnits, upperUnits, tariffMode, flatRate, tierLimitKwh, baseRatePerKwh, highRatePerKwh }	503 prediction service unavailable; 422 insufficient history or prediction rejected.

The Express API loads the user's usage and saved tariff, forwards them to FastAPI, and saves the returned estimate for the next invoice cycle. A repeated forecast for the same user and cycle updates the existing prediction row.

Error handling

- 401 means the bearer token is missing, expired, or invalid; the client clears a saved token on authenticated 401 responses.
- 400 indicates invalid request fields or date boundaries; 404 indicates a missing user-owned record or an unknown route.
- 409 indicates an email collision. 422 indicates forecast data is insufficient or rejected by the prediction service. 503 means that service did not respond successfully within the timeout.
- Unexpected Express errors return a generic 500 JSON message; server logs include the error class rather than the request payload.

Data model and usage transformations

The database separates account configuration, calendar readings, weekly summaries, generated forecasts, and alert messages. User-owned records carry a user_id so route queries can enforce ownership in SQL.

Table / entity	Key fields	Role
users	user_id; unique email; password_hash; tariff_mode; price_per_kwh; invoice_day	Credentials and invoice/tariff preferences. Password hash is excluded from normal entity selection.
daily_consumption_entries	user_id + entry_date unique; units_consumed; reading_type	Direct daily-use readings or cumulative meter totals.
consumption_records	user_id + week_start unique; units_consumed; bill_amount	Weekly summaries keyed to Monday. Legacy weekly interface remains available.
prediction_results	user_id + prediction_month unique; predicted_units; predicted_bill	Most recent persisted prediction for a user and invoice cycle.
alerts	alert_id; user_id; alert_type; message; created_at	Stored user alert history.

How daily readings become weekly history

- Direct daily-use entries contribute their entered kWh to that calendar week.
- Meter-total entries contribute the non-negative difference from the previous meter total. The first total is a baseline and has no derived usage.
- Daily usage is summed by Monday-based week and rounded to two decimals.
- When at least one daily entry exists for a user, its computed weekly total replaces that week's weekly summary. Other weeks continue to use weekly summary records. If no daily entries exist, weekly records supply the history.
- That unified history feeds the dashboard, alert evaluation, monthly grouping, and prediction request.

Dashboard calculations

The dashboard reports average weekly units, the latest two-week comparison, an above-average signal, alerts, saved predictions, and energy-saving tips. The alert rule requires at least three earlier weeks and fires when the latest week is more than 20% above their average. Editing or deleting a weekly record recalculates the stored above-average alert state.

Monthly summaries assign a week to the invoice cycle that contains its Monday start. Invoice day is configurable from 1 through 28. A month with fewer than four weekly records is marked partial; month comparisons are unavailable where the implementation lacks comparable values.

Prediction service flow and calculation

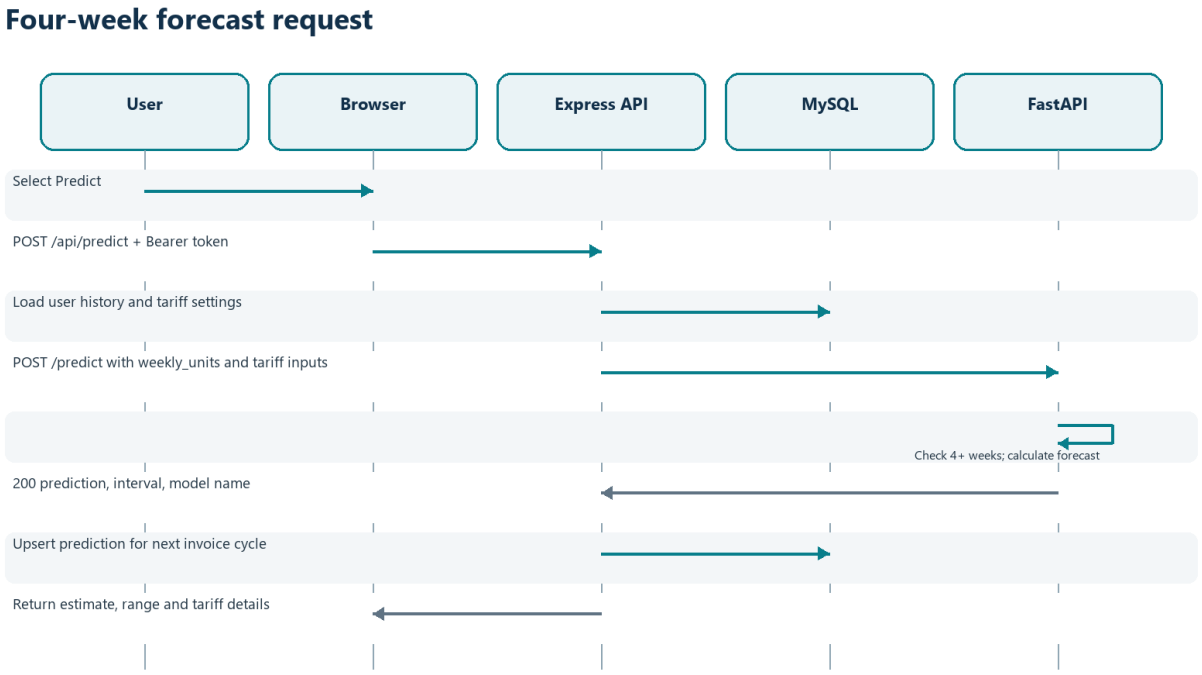


Figure 4. Forecast requests stay behind the Express API; the browser never calls FastAPI directly.

Internal service contract

```
POST {ML_SERVICE_URL}/predict
Content-Type: application/json
{
  "weekly_units": [ ... oldest first ... ],
  "price_per_kwh": null,
  "high_rate_per_kwh": 0.30,
  "tier_limit_kwh": 6000
}
```

FastAPI validates the request with Pydantic. Weekly usage must be non-negative and the model requires at least four observations. Insufficient history is returned as HTTP 422. Express applies a five-second timeout and maps connection/timeout failure to HTTP 503.

Current forecast algorithm

- Use the latest four weekly observations and calculate their arithmetic mean.
- Repeat that mean for four future weeks, then sum those four values as the next-month estimate.
- Calculate the bill using a progressive default tariff: the first 6,000 kWh at 0.18 SAR/kWh and excess usage at 0.30 SAR/kWh. If a flat override is saved, it replaces the tier schedule.
- Build a lower and upper estimate using the nearest evaluated baseline MAE for history lengths 8, 16, or 26 weeks; the lower bound cannot be below zero.
- Return model name last_four_week_mean and the four projected weekly values. The configured scikit-learn dependency does not mean this live path uses a Random Forest; the current function uses the recent-four-week mean.

Result persistence and time window

Express computes the next invoice cycle date from the current UTC date and the user's invoice day, then upserts one prediction result per user and prediction month. This persisted history appears in subsequent dashboard responses. If fewer than four weekly records are present, the prediction endpoint fails with 422 and the UI displays the returned error.

Security, deployment, and maintenance

Security controls in the request path

Control	Implementation
Transport and headers	Helmet security headers are enabled by Express. The client uses same-origin relative API paths; production deployment should provide TLS at the serving edge.
Authentication	JWT subject identifies the user; tokens expire after seven days. Protected routes verify bearer tokens before database access.
Ownership	SQL operations filter by authenticated user_id, including read, update, and delete operations.
Credential storage	Passwords are bcrypt-hashed; login errors do not reveal whether the account exists.
Request limits	Authentication routes are limited to 10 requests per 15 minutes. JSON body size is capped at 10 KB.
Browser token storage	The client stores JWT as se_token in localStorage. Injected scripts could read it; the repository README recommends a strict content security policy before public deployment.
Configuration	Production requires a JWT secret of at least 32 characters. DB and ML service locations are environment-configured.

Docker Compose runtime

The Compose stack contains MySQL 8, the FastAPI service, and the Express web app. The web service waits for healthy database and prediction containers. Startup runs compiled TypeORM migrations and then starts Express on port 3000. MySQL data lives in the mysql_data named volume, so stopping containers does not remove the database.

```
Copy .env.example to .env and set DB_PASSWORD and JWT_SECRET
docker compose up -d --build
Open http://localhost:3000
```

Schema and change management

TypeORM migrations are the only schema definition and upgrade path; synchronize is false. For a local installation, configure web/.env, provision the database, then run the migration command before starting the application. Back up an existing database before applying new migrations. The repository also includes analytics and API tests; API integration tests require a dedicated test database.

Implementation map

Area	Source location
Express setup and middleware	web/src/app.ts; web/src/server.ts
Authentication and JWT	web/src/routes/auth.ts; web/src/auth.ts
Daily and weekly records	web/src/routes/consumption.ts
Dashboard, settings, alerts, prediction orchestration	web/src/routes/insights.ts; web/src/analytics.ts
Browser API and dashboard behavior	web/client/api.ts; web/client/auth.ts; web/client/dashboard.ts
Prediction HTTP contract and model	ml-service/app.py; ml-service/model.py; ml-service/tariff.py
Entities and migrations	web/src/entities/; web/migrations/; web/src/data-source.ts
Container topology	compose.yml; web/Dockerfile; ml-service/Dockerfile